PHOSKEY 光子精密

GL 系列 3D 相机 通信库

一参考手册

V1.6.35

2026.02

www.phoskey.com

目录

1 运行环境 1

1.1 运行环境 1

2 文件构成 1

3 嵌入方法 1

3.1 C++ 引入动态链接库 1

3.2 C#引入动态链接库 2

4 变量类型 2

5 结构体定义 3

5.1 GLX8_2_Data 3

5.2 GLX8_2_ModelInfo 3

5.3 GLX8_2_ETHERNET_CONFIG 3

5.4 GLX8_2_STR_CALLBACK_INFO 4

5.5 GLX8_2_ModelInfo 4

5.6 GLX8_2_ImageData 5

5.7 GLX8_2_CloudData 5

5.8 返回码定义 5

6 函数 7

6.1 函数一览 7

6.2 函数参考 9

7 软件开发接口函数调用顺序 30

7.1 设置获取参数 30

7.2 单轮廓测量 34

7.3 阻塞方式获取数据 37

© 2025 光子（深圳）精密 版权所有

7.4 非阻塞方式有限循环获取数据 40
7.5 非阻塞方式无限循环 45
7.6 回调方式 50

© 2025 光子（深圳）精密 版权所有

1 运行环境

操作系统	Windows10 32/64 位系统（Professional/Enterprise） Windows8.1 32/64 位系统（Professional/Enterprise） Windows7 32/64 位系统（SP1 或更高版本）（Professional/Ultimate） Windows11 32/64 位系统（Professional/Enterprise）
CPU	Corei32.3GHz 或更快的处理器
内存容量	8GB 以上
硬盘	10GB 以上
接口	Ethernet1000BASE-T
显卡	2g 显存

1.1 运行环境

以下内容是在使用 GL 系列 3D 相机通信库执行应用程序时的基本环境。

1.1.1 Microsoft C Runtime Library s

这是执行 DLL 所必需的运行环境。

1.1.2 Microsoft .NET Framework

这是执行示例程序所必需的运行环境。

2 文件构成

gl_linelaser_sdk.dll	DLL 本体，依赖库见链接库
gl_linelaser_sdk.lib	gl_linelaser_sdk.dll 的引入库
phoskey_ss.h	定义通信接口的头文件

3 嵌入方法

3.1 C++ 引入动态链接库

- gl_linelaser_sdk.dll
- filterGrad.dll
- opencv_core4.dll
- zlib1.dll
- opencv_world470.dll
- gl_linelaser_sdk.lib

© 2025 光子（深圳）精密 版权所有

- opencv_imgproc4.dll

3.1.1 链接

明链接、暗链接均可使用。进行暗链接时，请链接“gl_linelaser_sdk.lib”。

3.1.2 包含文件

请务必将下列头文件放置在源文件中

- phoskey_ss.h/Phoskey3DLineLaser.cs

3.2 C#引入动态链接库

- gl_linelaser_sdk.dll
- opencv_world470.dll
- filterGrad.dll
- opencv_core4.dll
- zlib1.dll
- opencv_imgproc4.dll

3.2.1 引入库文件

- phoskey_ss.cs/Phoskey3DLineLaser.cs

4 变量类型

unsigned char(uint8_t)	无符号的 8bit 整数
unsigned short(uint16_t)	无符号的 16bit 整数
Unsigned long(uing32_t)	无符号的 32bit 整数
Int(int32_t)	有符号的 32bit 整数
unsigned int (uint32_t)	无符号的 32bit 整数
long long	有符号的 64bit 整数
double	有符号的 64bit 浮点数

© 2025 光子（深圳）精密 版权所有

5 结构体定义

5.1 GLX8_2_Data

名称	机器 ID GLX8_2_Data

定义	struct GLX8_2_Data
	{
	int lDeviceId;
	int Head;
	};

描述	机器 id 信息
备注	-

5.2 GLX8_2_ModelInfo

名称	当前设备信息 GLX8_2_ModelInfo

定义	struct GLX8_2_ModelInfo{
	char Model[16]; // 型号
	int PointSetCount; // 设置的批处理数量
	int ProfileDataWidth; // 数据宽度
	double xPixth; // x 方向点间距 mm
	double yPixth; // 时间轴间距 mm， -1 为没有设置，需要自行设置
	char HeaderSerial[32]; // 传感器头序列号
	char CtrlSerial[32]; // 控制器序列号
	int CurrentEncoder; // 当前编码器的值
	int CurrentBatchTimes; // 当前批处理次数
	char dllVersion[16]; // dll 库的版本号
	double zRangmin; // z 最小量程 mm
	double zRangmax; // z 最大量程 mm
	double xRangmin; // x 最小量程 mm
	double xRangmax; // x 最大量程 mm
	double res_z; //高度分辨率
	};

描述	机器信息
备注	-

5.3 GLX8_2_ETHERNET_CONFIG

名称	地址配置结构体 GLX8_2_ETHERNET_CONFIG
定义	struct GLX8_2_ETHERNET_CONFIG {

© 2025 光子（深圳）精密 版权所有

	unsigned char abyIpAddress[4]; };
描述	连接控制器的 IP 地址。 地址为 192.168.1.10 时， 设定为 GLX8_2_ETHERNET_CONFIG Net; Net.abyIpAddress[0]=192 Net.abyIpAddress[1]=168 Net.abyIpAddress[2]=1 Net.abyIpAddress[3]=10
备注	-

5.4 GLX8_2_STR_CALLBACK_INFO

名称	回调信息结构体 GLX8_2_STR_CALLBACK_INFO

定义	struct
	{
	int xPoints; //x 方向数据数量
	int BatchPoints; //批处理数量
	int BatchTimes; //批处理次数
	double xPixth; //x 方向点间距
	int startEncoder; //批处理开始编码器值
	int HeadNumber; //相机头数量
	int returnStatus; //0:正常批处理
	int EXTROI_EN; // 0=正常批处理，1=yroi 行数会与设置的批处理数量不一致
	};

描述	
备注	-

5.5 GLX8_2_ModelInfo

名称	相机机型信息结构体 GLX8_2_ModelInfo

定义	struct GLX8_2_ModelInfo
	{
	char Model[16]; // 型号
	int PointSetCount; // 设置的批处理数量
	int ProfileDataWidth; // 数据宽度
	double xPixth; // x 方向点间距 mm
	double yPixth; // 时间轴间距 mm， -1 为没有设置，需要自行设置
	char HeaderSerial[32]; // 传感器头序列号
	char CtrlSerial[32]; // 控制器序列号

© 2025 光子（深圳）精密 版权所有

	int CurrentEncoder; // 当前编码器的值
	int CurrentBatchTimes; // 当前批处理次数
	char dllVersion[16]; // dll 库的版本号
	double zRangmin; // z 最小量程 mm
	double zRangmax; // z 最大量程 mm
	double xRangmin; // x 最小量程 mm
	double xRangmax; // x 最大量程 mm
	double res_z; //高度分辨率
	};

描述	
备注	-

5.6 GLX8_2_ImageData

名称	点云转换输入信息结构体 GLX8_2_ImageData

定义	struct _GLX8_2_ImageData
	{
	int width, height;
	float origin_x = 0, origin_y = 0, origin_z = 0;
	int* profile_data = nullptr;
	unsigned char* gray_data = nullptr;
	
	char reserved[64];
	};

描述	
备注	-

5.7 GLX8_2_CloudData

名称	点云数据输出结构体 GLX8_2_CloudData

定义	struct _GLX8_2_CloudData {
	float x, y, z;
	unsigned char gray;
	};

描述	
备注	-

5.8 返回码定义

定义	enum class GLX8_2_Rc
	{
	Ok = 0, // 正常返回
	SocketErr = 1001, //通信异常
	SystemErr = 3090, // 系统错误

© 2025 光子（深圳）精密 版权所有

	SystemCaliErr = 3091, // 系统错误
	ErrHaveOpenDeviceYet = 4091, //设备已经打开
	ErrOpenDevice = 4096, // 打开设备失败
	ErrNoDevice = 4097, // 设备不存在
	ErrSend = 4098, //发送失败
	ErrReceive = s, // 接收失败
	ErrTimeout = 4100, // 超时
	ErrNomemory = 4101, // 内存错误
	ErrParameter = 4102, // 参数错误
	ErrRecvFmt = 4103, // 接收格式错误
	ErrHispeedNoDevice = 4105,
	ErrHispeedOpenYet = 4106,
	ErrHispeedRecvYet = 4107,
	ErrBufferShort = 4108, // 缓存太小
	ErrDisallowedSettings = 5001, // 不允许的设置
	ErrNullData = 5002, // 批处理数据为空
	ErrBatchTop = 5003, // 错误的开始行数
	ErrBatchFrame=5004,//批处理帧错误
	ErrBatchon=5005,//没有开启批处理
	ErrVersion = 5100, // 固件版本与 sdk 版本不匹配
	ERRCodeerSpeed = 6001, // 编码器过快
	ErrBatch = 0x8081, // 不允许批处理
	ErrSetting = -1, // 错误的调用,如 ID 冲突、未采集完行数再次触发
	ErrBatchStopped=-100 //批处理已停止
	};

备注	返回的是 int 整数，可以转为 GLX8_2_Rc 枚举

© 2025 光子（深圳）精密 版权所有

6 函数

6.1 函数一览

6.1.1 系统控制

函数名	概要
GLX8_2_Initialize	初始化设备
GLX8_2_GetVersion	获取版本号

6.1.2 相机参数配置/设置

函数名	概要
GLX8_2_SetSetting	设置配置
GLX8_2_GetSetting	读取配置
GLX8_2_GetModels	获取相机型号
GLX8_2_GetHeaderSerial	获取相机头序列号
GLX8_2_GetModelInfos	获取传感器的型号以及测量相关的数据
GLX8_GetActiveProgram	获取当前配方号
GLX8_2_SET_X_ROI	设置 X 轴 ROI
GLX8_2_GET_X_ROI	获取 X 轴 ROI
GLX8_2_SetXpitch	设置 X 轴间距
GLX8_2_GetXpitch	获取 X 轴间距
L42_ExportParameters	参数导出
L42_LoadParameters	参数载入

6.1.3 设备连接相关通信

函数名	概要
GLX8_2_SearchOnline	搜索在线设备
GLX8_2_EthernetOpen	通过 Ethernet 建立连接
GLX8_2_CommClose	断开连接

6.1.4 测量控制

函数名	概要
GLX8_2_StartMeasure	开始有限循环批处理(软件触发)
GLX8_2_StartIOTriggerMeasure	开始有限循环批处理(IO 触发)
GLX8_2_StopMeasure	结束有限/无限循环批处理
GLX8_2_SwitchProgram	切换相机配置的参数
GLX8_2_StartMeasureWithCallback	回调模式开始批处理（可选软触发 IO 触发）
GLX8_2_StartBatchRollData	开始无限循环批处理（软件触发）

© 2025 光子（深圳）精密 版权所有

GLX8_2_StartIOBatchRollData	开始无限循环批处理（IO 触发）

6.1.5 测量结果获取

函数名	概要
GLX8_2_GetSingleProfile	取当前一条轮廓
GLX8_2_GetCurrentEncoder	获取当前编码器的值
GLX8_2_ProfilePointSetCount	当前批处理设置定行数
GLX8_2_ProfilePointCount	阻塞获取批处理实际获取行数
GLX8_2_ProfileDataWidth	获取数据宽度
GLX8_2_ProfileData_XPitch	获取数据 x 方向间距
GLX8_2_ProfileData_YPitch	获取数据 y 方向间距

6.1.6 阻塞获取批处理数据

函数名	概要
GLX8_2_ReceiveData	阻塞等待批处理数据
GLX8_2_GetEncoder	阻塞获取编码器值
GLX8_2_GetProfileData	阻塞获取轮廓数据
GLX8_2_GetIntensityData	阻塞获取亮度数据
GLX8_2_GetTimestamps	阻塞获取每行的时间
GLX8_2_GetBatchTimes	获取当前批处理次数

6.1.7 连续获取数据

函数名	概要
GLX8_2_GetProfileContiuneData	连续获取轮廓数据
GLX8_2_GetIntensityContiuneData	连续获取亮度数据
GLX8_2_GetEncoderContiune	连续获取编码器数据

6.1.8 无限循环获取数据

函数名	概要
GLX8_2_GetBatchRollData	无终止循环获取数据，需要开启批处理测量
GLX8_2_GetBatchRollData_Ex	无终止循环获取数据，需要开启批处理测量，支持获取时间戳，批处理次数
GLX8_2_SetBatchRollProfilePoint	无终止循环设定行数

6.1.9 回调获取数据

函数名	概要
GLX8_2_SetBatchOneTimeDataHandler	设置回调函数,建议获取数据后另外开启线程进行处理（获取数据模式:批处理一次回调一次）

© 2025 光子（深圳）精密 版权所有

GLX8_2_SetBatchOneTimeDataHandler_V2	设置回调函数,建议获取数据后另外开启线程进行处理（获取数据模式:批处理一次回调一次）
GLX8_2_GetBatchProfilePoint	批处理轮廓获取（获取数据模式:批处理一次回调一次）
GLX8_2_GetBatchProfilePoint16	16 位批处理轮廓获取(获取数据模式:批处理一次回调一次)
GLX8_2_GetBatchIntensityPoint	批处理亮度获取（获取数据模式:批处理一次回调一次）
GLX8_2_GetBatchEncoderPoint	批处理编码器获取（获取数据模式:批处理一次回调一次）
GLX8_2_GetBatchTimestamps	批处理时间戳 单位 us（获取数据模式:批处理一次回调一次）
GLX8_2_ImageDataToPointCloud	点云数据格式转换

6.2 函数参考

6.2.1 GLX8_2_Initialize

格式	`int CPP_STDCALL GLX8_2_Initialize();`
参数	-
返回值	参考第 5 章函数返回码定义
说明	查找相机时需要初始化 SDK
运行用时	约 1ms
支持版本	1.4.17

6.2.2 GLX8_2_GetVersion

格式	`char* CPP_STDCALL GLX8_2_GetVersion();`
参数	-
返回值	返回 C 语言类型字符串
说明	获取 sdk 版本号
运行用时	约 1ms
支持版本	1.4.17

6.2.3 GLX8_2_SearchOnline

格式	`GLX8_2_ETHERNET_CONFIG* CPP_STDCALL GLX8_2_SearchOnline(int* count, int timeout);`
参数	● count 获取到的设备数量 ● timeout 超时时间 ms 调用前需要先调用 GLX8_2_Initialize 初始化
返回值	参考结构体定义
说明	查找所有在线设备
运行用时	约 200ms

© 2025 光子（深圳）精密 版权所有

支持版本	1.4.17

6.2.4 GLX8_2_EthernetOpen

格式	int CPP_STDCALL GLX8_2_EthernetOpen(int lDeviceId, GLX8_2_ETHERNET_CONFIG * pEthernetConfig);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● pEthernetConfig 参考结构体定义
返回值	参考第 5 章函数返回码定义
说明	建立连接，使设备能够与通过 Ethernet 连接的控制器进行通信
运行用时	约 200ms
支持版本	1.4.17

6.2.5 GLX8_2_CommClose

格式	int CPP_STDCALL GLX8_2_CommClose(int lDeviceId);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255
返回值	参考第 5 章函数返回码定义
说明	断开连接
运行用时	约 1ms
支持版本	1.4.17

6.2.6 GLX8_2_SetSetting

格式	int CPP_STDCALL GLX8_2_SetSetting(uint32_t lDeviceId, int Depth, int Type, int Category, int Item, int* Target, char* pData, int DataSize);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● Depth：(暂未启用)写 1 写入内存设定 写入 2 永久保存 ● Type：(仅对永久保存生效) 0x10-0x4Fh 0-63 配方 ● Category 如需发送 / 接收程序 0 至 15 的设定，请像触发设定、拍摄设定一样，指定发送 / 接收哪一项设定。发送 / 接收环境设定、测量通用设定时，指定为 0。 ● Item 指定发送 / 接收 Category 指定项目中的哪一项设定 ● Target 默认 0 ● pData pData 指定用于存储所发送设定数据的缓存

© 2025 光子（深圳）精密 版权所有

<table>
<tr><td></td><td>

- DataSize

dwDataSize 发送设定数据的 BYTE 单位大小。
</td></tr>
<tr><td>返回值</td><td>参考第 5 章函数返回码定义</td></tr>
<tr><td>说明</td><td>
参数设定。

- Depth(暂未启用,默认 0x01 即可)：

0x01 ：将参数写入非保存区域，参数生效，但关闭电源后设定将不被保存。（重新启动时，保存区域的设定生效）

0x02：将参数写入保存区域，参数生效，关闭电源后设定仍将被保存到控制器。

- Type(仅对永久保存生效，默认 0x10 即可)：

-1：返回当前配方的设定；

0x10h 至 0x4Fh：配方 0 至配方 63 中的哪一项设定。

- Category：指定参数页，0：触发设定页面；1：拍摄设定页面；2：轮廓设定页面。
- Item：指定发送 / 接收 Category 指定项目中的哪一项设定。
- Target：指定修改目标，C++填入 nullptr,C#填入 IntPtr.Zero。
- pData：写入的具体数据。
- DataSize：写入数据的长度。

<table>
<tr><th>支持项目</th><th>Category</th><th>Item</th><th>Target</th><th>pData</th><th>DataSize</th></tr>
<tr><td>触发模式</td><td>0x00</td><td>0x01</td><td>nullptr</td><td>0：连续触发
2：编码器触发</td><td>4</td></tr>
<tr><td>批处理开关</td><td>0x00</td><td>0x03</td><td>nullptr</td><td>0：
批处理 OFF
1：
批处理 ON</td><td>4</td></tr>
<tr><td>编码器输入模式</td><td>0x00</td><td>0x07</td><td>nullptr</td><td>0：1 相 1 递增；
1：2 相 1 递增；
2：2 相 2 递增；
3：2 相 4 递增；</td><td>4</td></tr>
<tr><td>细化点数</td><td>0x00</td><td>0x09</td><td>nullptr</td><td>0-9999</td><td>4</td></tr>
<tr><td>批处理点数</td><td>0x00</td><td>0x0A</td><td>nullptr</td><td>1~50000</td><td>4</td></tr>
<tr><td>批处理输出类型</td><td>0x00</td><td>0x0B</td><td>nullptr</td><td>0：轮廓
1：轮廓+亮度</td><td>4</td></tr>
<tr><td>同步功能</td><td>0x00</td><td>0x20</td><td>nullptr</td><td>0：关
1：开
2：主机
3：从机</td><td>4</td></tr>
<tr><td>自定以采样频率</td><td>0x00</td><td>0x22</td><td>nullptr</td><td>具体值（小于最大采样率）</td><td>4</td></tr>
<tr><td>采样频率</td><td>0x00</td><td>0x23</td><td>nullptr</td><td>0：最大采样频率</td><td>4</td></tr>
</table>
</td></tr>
</table>

© 2025 光子（深圳）精密 版权所有

	使能				1：自定义采样频率	
	开关电平信号	0x00	0x26	nullpt	0:关闭 1:开启	4
	获取最大采样频率	0x00	0xff	nullptr	最大采样率（仅读）	4
	Binning	0x01	0x02	nullptr	0:关闭 1:开启	4
	Z 方向测量范围	0x01	0x03	nullptr	0：FULL 1：1/2 2：1/4 3：1/8	4
	单帧 HDR	0x01	0x05	nullptr	0：高精度 1：高动态范围 1 2：高动态范围 2 3：高动态范围 3 4：高动态范围 4 5：高动态范围 5 6：高动态范围 6 7：高动态范围 7 8：高动态范围 8 9：高动态范围 9	4
	峰值选择	0x01	0x11	nullptr	2：标准 0：近端 1：远端 7：连续 6：连续最大 4：连续近端 5：连续远端 3：使之转化为无效数据	4
	低阈值	0x01	0x12	nullptr	具体值	4
	高阈值	0x01	0x13	nullptr	具体值	4
	光晕过滤	0x01	0x14	nullptr	0：关闭 1：光晕 1 2：光晕 2 3：光晕 3 4：自定义	4
	自定义光晕过滤最小宽度	0x01	0x15	nullptr	具体值	4
	自定义光	0x01	0x16	nullptr	具体值	4

© 2025 光子（深圳）精密 版权所有

	晕过滤最大宽度					
	自定义光晕过滤宽高比	0x01	0x17	nullptr	具体值	4
	增益	0x01	0x20	nullptr	0：1 挡 1：2 挡 2：3 挡 3：4 挡	4
	曝光时间	0x01	0x21	nullptr	具体值（us）	4
	死角处理	0x02	0x05	nullptr	0：设为无效值 1：直线补间 2：垂直补间	4
	死角处理高级梯度阈值	0x02	0x13	nullptr	具体值(float)	4
	死角处理高级高度阈值	0x02	0x14	nullptr	具体值	4
	左右反转	0x02	0x06	nullptr	0：关闭 1：开启	4
	上下反转	0x02	0x07	nullptr	0：关闭 1：开启	4
	X 轴中位数滤波	0x02	0x0A	nullptr	1：OFF 3；3 点 5：5 点 7：7 点 9：9 点	4
	X 轴平均数滤波	0x02	0x0B	nullptr	1：1（OFF） 4；4 点 8：8 点 16：16 点 32：32 点 64：64 点 128：128 点 256：256 点	4
	时间轴中位滤波	0x02	0x0C	nullptr	1：OFF 3；3 点 5：5 点 7：7 点 9：9 点	4

© 2025 光子（深圳）精密 版权所有

	<table><tr><td>时间轴平均滤波</td><td>0x02</td><td>0x0D</td><td>nullptr</td><td>1：1（OFF）
	3：3 点
	5：5 点
	7：7 点
	9：9 点</td><td>4</td></tr></table>
	举例：
	触发设定：
	int data;
	int depth = 0x01;//留用 默认 0x01 即可
	int Type = 0x10;//留用 默认 0x10 即可
	连续触发： data = 0x00; GLX8_2_SetSetting(0, depth, Type, 0, 0x01, nullptr, data, 4);
	编码器触发：data = 0x02; GLX8_2_SetSetting(0, depth, Type, 0, 0x01, nullptr, data, 4);
	批处理设定：
	批处理关闭：data = 0x00; GLX8_2_SetSetting(0, depth, Type, 0, 0x03, nullptr, data, 4);
	批处理使能：data = 0x01; GLX8_2_SetSetting(0, depth, Type, 0, 0x03, nullptr, data, 4);
	永久保存额外两步:
	1.Type:设置程序编号 0x10 -0 编号以此类推
	2.尾部添加 GLX8_2_SetSetting(0, 2, type, 0, 0, 0, 0, 0);

运行用时	约 1ms
支持版本	1.4.17

6.2.7 GLX8_2_GetSetting（参数同 GLX8_2_SetSetting）

格式	int CPP_STDCALL GLX8_2_GetSetting(uint32_t lDeviceId, int Type, int Category, int Item, int* Target, char* pData, int DataSize);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● 其他同上 GLX8_2_SetSetting
返回值	参考第 5 章函数返回码定义
说明	读取配置
运行用时	**约 1ms**
支持版本	1.4.17

6.2.8 GLX8_2_GetModels

格式	char* CPP_STDCALL GLX8_2_GetModels(int lDeviceId)
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255
返回值	返回 c 语言类型字符串

© 2025 光子（深圳）精密 版权所有

说明	获取相机型号
运行用时	约 1ms
支持版本	1.4.17

6.2.9 GLX8_2_GetHeaderSerial

格式	char* CPP_STDCALL GLX8_2_GetHeaderSerial(int lDeviceId, int Head);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255
返回值	返回 c 语言类型字符串
说明	获取相机头序列号
运行用时	约 1ms
支持版本	1.4.17

6.2.10 GLX8_2_GetModelInfos

格式	int CPP_STDCALL GLX8_2_GetModelInfos(int lDeviceId, GLX8_2_ModelInfo* infos);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● infos GLX8_2_GetModelInfos 参考结构体定义
返回值	参考第 5 章函数返回码定义
说明	获取传感器的型号以及测量相关的数据
运行用时	约 1ms
支持版本	1.4.17

6.2.11 GLX8_GetActiveProgram

格式	int GLX8_GetActiveProgram(int lDeviceId, char* pbyProgramNo);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● pbyProgramNo 当前配方号
返回值	参考第 5 章函数返回码定义
说明	获取传感器的型号以及测量相关的数据
运行用时	约 1ms
支持版本	1.4.17

6.2.12 GLX8_2_SET_X_ROI

格式	int CPP_STDCALL GLX8_2_SET_X_ROI(int lDeviceId, int start, int end);
参数	● lDeviceId

© 2025 光子（深圳）精密 版权所有

	指定通过哪台通信设备进行通信，范围 0-255 ● start 起始 x 坐标 ● end 结束 x 坐标
返回值	参考第 5 章函数返回码定义
说明	设置 x 轴 Roi 开始像素到结束像素
运行用时	**约 1ms**
支持版本	1.4.17

6.2.13 GLX8_2_GET_X_ROI

格式	`int CPP_STDCALL GLX8_2_GET_X_ROI(int lDeviceId, int* start, int* end);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 start 起始 x 坐标 ● end 结束 x 坐标
返回值	参考第 5 章函数返回码定义
说明	获取 x 轴 Roi 开始像素到结束像素
运行用时	**约 1ms**
支持版本	1.4.17

6.2.14 GLX8_2_SetXpitch

格式	`int CPP_STDCALL GLX8_2_SetXpitch(int lDeviceId, uint32_t dwXpitch);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● dwXpitch 单位：0.01um
返回值	参考第 5 章函数返回码定义
说明	设置 X 轴间距
运行用时	**约 1ms**
支持版本	1.4.17

6.2.15 GLX8_2_GetXpitch

格式	`int CPP_STDCALL GLX8_2_GetXpitch(int lDeviceId, uint32_t* dwXpitch);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● dwXpitch 单位：0.01um
返回值	参考第 5 章函数返回码定义

© 2025 光子（深圳）精密 版权所有

说明	获取 X 轴间距
运行用时	**约 1ms**
支持版本	1.4.17

6.2.16 GLX8_2_ExportParameters

格式	`const char* CPP_STDCALL GLX8_2_ExportParameters(int lDeviceId, unsigned int* size);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● size 参数大小
返回值	导出参数头
说明	导出当前配置参数
运行用时	**约 1ms**
支持版本	1.6.35

6.2.17 GLX8_2_LoadParameters

格式	`int CPP_STDCALL GLX8_2_LoadParameters(int lDeviceId, const char* settings, unsigned int size);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● settings 参数配置 ● size 参数大小
返回值	导出参数头
说明	导出当前配置参数
运行用时	**约 1ms**
支持版本	1.6.35

6.2.18 GLX8_2_StartMeasure

格式	`int CPP_STDCALL GLX8_2_StartMeasure(int lDeviceId, int Timeout);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● Timeout 超时时间
返回值	参考第 5 章函数返回码定义
说明	软件批处理测量开始
运行用时	约 1ms
支持版本	1.4.17

© 2025 光子（深圳）精密 版权所有

6.2.19 GLX8_2_StartIOTriggerMeasure

格式	int CPP_STDCALL GLX8_2_StartIOTriggerMeasure(int lDeviceId, int Timeout, int restart);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● Timeout 等待 io 超时时间,0 无限等待 ● Restart 预留设为 0
返回值	参考第 5 章函数返回码定义
说明	软件批处理测量开始
运行用时	约 1ms
支持版本	1.4.17

6.2.20 GLX8_2_StopMeasure

格式	int CPP_STDCALL GLX8_2_StopMeasure(int lDeviceId)
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255
返回值	参考第 5 章函数返回码定义
说明	软件批处理测量结束
运行用时	约 1ms
支持版本	1.4.17

6.2.21 GLX8_2_SwitchProgram

格式	int CPP_STDCALL GLX8_2_SwitchProgram(int lDeviceId, int No);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● No 程序设定编号 0-编号 0 1-编号 1
返回值	参考第 5 章函数返回码定义
说明	切换相机配置参数
运行用时	约 1ms
支持版本	1.4.17

6.2.22 GLX8_2_StartMeasureWithCallback

格式	int CPP_STDCALL GLX8_2_StartMeasureWithCallback(int lDeviceId, int ImmediateBatch);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● ImmediateBatch

© 2025 光子（深圳）精密 版权所有

	0:立即开始批处理　非 0:等待外部开始批处理.
返回值	参考第 5 章函数返回码定义
说明	开始批处理（获取数据模式:批处理一次回调一次） 回调方式的开始批处理函数。在第 1 次批处理前调用 1 次即可，后续的数据获取流程中无需再调用。
运行用时	**约 1ms**
支持版本	1.4.17

6.2.23 GLX8_2_StartBatchRollData

格式	`int CPP_STDCALL GLX8_2_StartBatchRollData(int lDeviceId);`
参数	● lDeviceId 指定通过哪台通信设备进行无限循环批处理，范围 0-255
返回值	参考第 5 章函数返回码定义
说明	软触发无限循环批处理
运行用时	**约 1ms**
支持版本	1.4.17

6.2.24 GLX8_2_StartIOBatchRollData

格式	`int CPP_STDCALL GLX8_2_StartIOBatchRollData(int lDeviceId);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255
返回值	参考第 5 章函数返回码定义
说明	硬触发无限循环批处理
运行用时	**约 1ms**
支持版本	1.4.17

6.2.25 GLX8_2_GetSingleProfile

格式	`int CPP_STDCALL GLX8_2_GetSingleProfile(unsigned int lDeviceId, int* pProfileData, unsigned int* pEncoder)`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● pProfileData 轮廓数据指针　int32_t[4096]（8000series） ● pEncoder　当前编码器的指针，int ec = *pEncoder　;
返回值	参考第 5 章函数返回码定义
说明	获取当前一条轮廓,需要关闭批处理测量
运行用时	**约 1ms**
支持版本	1.4.17

© 2025 光子（深圳）精密 版权所有

6.2.26 GLX8_2_GetCurrentEncoder

格式	int CPP_STDCALL GLX8_2_GetCurrentEncoder(int lDeviceId, GLX8_2_Data DataObj);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr
返回值	当前编码器的值
说明	获取当前编码器的值
运行用时	约 1ms
支持版本	1.4.17

6.2.27 GLX8_2_ProfilePointSetCount

格式	int CPP_STDCALL GLX8_2_ProfilePointSetCount(int lDeviceId, GLX8_2_Data DataObj);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr
返回值	返回当前批处理设定行数，小于零失败
说明	获取当前批处理设定行数
运行用时	约 1ms
支持版本	1.4.17

6.2.28 GLX8_2_ProfilePointCount

格式	int CPP_STDCALL GLX8_2_ProfilePointCount(int lDeviceId, GLX8_2_Data DataObj);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr
返回值	返回当前批处理行数，小于零失败
说明	获取当前批处理行数，使用 GLX8_2_ReceiveData 后读取实际运行的行数
运行用时	约 1ms
支持版本	1.4.17

6.2.29 GLX8_2_ProfileDataWidth

格式	int CPP_STDCALL GLX8_2_ProfileDataWidth(int lDeviceId, GLX8_2_Data

© 2025 光子（深圳）精密 版权所有

	DataObj);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr
返回值	返回宽度像素，（gl8xxx=4096，gl7xxx=2048）
说明	获取数据宽度
运行用时	约 1ms
支持版本	1.4.17

6.2.30 GLX8_2_ProfileData_XPitch

格式	double CPP_STDCALL GLX8_2_ProfileData_XPitch(int lDeviceId, GLX8_2_Data DataObj);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr
返回值	返回数据 x 方向间距毫米单位(mm)
说明	获取数据 x 方向间距
运行用时	约 1ms
支持版本	1.4.17

6.2.31 GLX8_2_ProfileData_YPitch

格式	double CPP_STDCALL GLX8_2_ProfileData_YPitch(int lDeviceId, GLX8_2_Data DataObj);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr
返回值	返回数据 y 方向间距毫米单位(mm)，小于零为未开启 y 轴间距
说明	获取数据 y 方向间距
运行用时	约 1ms
支持版本	1.4.17

6.2.32 GLX8_2_ReceiveData

格式	int CPP_STDCALL GLX8_2_ReceiveData(int lDeviceId, GLX8_2_Data DataObj);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr

© 2025 光子（深圳）精密 版权所有

返回值	参考第 5 章函数返回码定义
说明	阻塞方式获取数据 通过后通过以下方式获取当前批处理数据 ● 获取批处理实际获取行数：GLX8_2_ProfilePointCount ● 高度数据获取：GLX8_2_GetProfileData ● 灰度数据获取：GLX8_2_GetIntensityData ● 编码器数据获取：GLX8_2_GetEncoder ● 批处理次数获取：GLX8_2_GetBatchTimes
运行用时	–
支持版本	-

6.2.33 GLX8_2_GetEncoder

格式	`int CPP_STDCALL GLX8_2_GetEncoder(int lDeviceId, GLX8_2_Data DataObj, uint32_t * Encoder);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr ● Encoder 数据缓存指针，需要分配大于接收到行数的内存 Encoder = new[GLX8_2_ProfilePointCount]
返回值	参考第 5 章函数返回码定义
说明	阻塞获取编码器值,使用 GLX8_2_ReceiveData 后使用
运行用时	–
支持版本	-

6.2.34 GLX8_2_GetProfileData

格式	`int CPP_STDCALL GLX8_2_GetProfileData(int lDeviceId, GLX8_2_Data DataObj, int32_t * Profile);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr ● Profile 数据缓存指针，需要分配大于接收到行数的内存 Profile = new[GLX8_2_ProfilePointCount * GLX8_2_ProfileDataWidth]
返回值	参考第 5 章函数返回码定义
说明	阻塞获取轮廓数据,使用 GLX8_2_ReceiveData 后使用
运行用时	–
支持版本	-

© 2025 光子（深圳）精密 版权所有

6.2.35 GLX8_2_GetIntensityData

格式	int CPP_STDCALL GLX8_2_GetIntensityData(int lDeviceId, GLX8_2_Data DataObj, uint8_t * Intensity);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr ● Intensity 数据缓存指针，需要分配大于接收到行数的内存 Intensity = new[GLX8_2_ProfilePointCount * GLX8_2_ProfileDataWidth]
返回值	参考第 5 章函数返回码定义
说明	阻塞获取亮度数据,使用 GLX8_2_ReceiveData 后使用
运行用时	–
支持版本	-

6.2.36 GLX8_2_GetTimestamps

格式	int CPP_STDCALL GLX8_2_GetTimestamps(int lDeviceId, GLX8_2_Data DataObj, uint64_t * timestamp);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr Timestamp 时间戳指针
返回值	参考第 5 章函数返回码定义
说明	获取时间戳
运行用时	约 1ms
支持版本	1.4.17

6.2.37 GLX8_2_GetProfileContiuneData

格式	int CPP_STDCALL GLX8_2_GetProfileContiuneData(int lDeviceId, GLX8_2_Data DataObj, int* profile, unsigned int count);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr ● profile 深度图指针 ● count 获取的最大数据长度

© 2025 光子（深圳）精密 版权所有

返回值	返回当前从上电开始的批处理总次数
说明	非阻塞获取轮廓数据
运行用时	**约 1ms**
支持版本	1.4.17

6.2.38 GLX8_2_GetIntensityContiuneData

格式	int CPP_STDCALL GLX8_2_GetIntensityContiuneData(int lDeviceId, GLX8_2_Data DataObj, int* intensity, unsigned int count);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr ● intensity 灰度图指针 ● count 获取的最大数据长度
返回值	返回当前从上电开始的批处理总次数
说明	非阻塞获取轮廓数据
运行用时	**约 1ms**
支持版本	1.4.17

6.2.39 GLX8_2_GetEncoderContiune

格式	int CPP_STDCALL GLX8_2_GetEncoderContiune(int lDeviceId, GLX8_2_Data DataObj, unsigned int* encoder, unsigned int count);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为 nullptr ● encoder 编码器数据指针 ● count 获取的最大数据长度
返回值	返回当前从上电开始的批处理总次数
说明	非阻塞获取轮廓数据
运行用时	**约 1ms**
支持版本	1.4.17

6.2.40 GLX8_2_GetBatchRollData

格式	int CPP_STDCALL GLX8_2_GetBatchRollData(int lDeviceId, const GLX8_2_Data DataObj,int* Profile, unsigned char* Intensity, unsigned int* Encoder, long long* FrameId, unsigned int* FrameLoss,unsigned int GetCnt);

© 2025 光子（深圳）精密 版权所有

参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为空 ● Profile 返回轮廓数据指针返回数据为 Int 型，单位为 0.01um ； 其中无效值以-9999*10⁵表示 ● Intensity 返回亮度数据指针 ● Encoder 返回编码器数据指针 ● FrameId 返回帧编码数据指针 ● FrameLoss 返回批处理过快掉帧数量数据指针 ● GetCnt 返回数据长度
返回值	<0 失败（参考第 5 章函数返回码定义） >0：实际返回的数据长度
说明	无终止循环获取数据
运行用时	**约 1ms**
支持版本	1.4.17

6.2.41 GLX8_2_GetBatchRollData_Ex

格式	int CPP_STDCALL GLX8_2_GetBatchRollData_Ex(int lDeviceId, const GLX8_2_Data DataObj,unsigned int* Batchtimes,int* Profile, unsigned char* Intensity, unsigned int* Encoder, long long* FrameId,unsigned long long* Timestamp, unsigned int* FrameLoss,unsigned int GetCnt);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● DataObj 预留为空 ● Batchtimes 返回批处理次数指针，单个值，单批数据属于同批处理次数 ● Profile 返回轮廓数据指针返回数据为 Int 型，单位为 0.01um ； 其中无效值以-9999*10⁵表示 ● Intensity 返回亮度数据指针 ● Encoder 返回编码器数据指针 ● FrameId 返回帧编码数据指针 ● Timestamp, 返回时间戳数据指针，单位 us，相机时间 ● FrameLoss 返回批处理过快掉帧数量数据指针 ● GetCnt 返回数据长度
返回值	<0 失败（参考第 5 章函数返回码定义） >0：实际返回的数据长度
说明	无终止循环设定行数
运行用时	**约 1ms**
支持版本	1.6.24

6.2.42 GLX8_2_SetBatchRollProfilePoint

格式	int CPP_STDCALL GLX8_2_SetBatchRollProfilePoint(int lDeviceId, const GLX8_2_Data DataObj, unsigned int points);
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255

© 2025 光子（深圳）精密 版权所有

	● DataObj 默认 nullptr ● points 设置中止行数， 0：表示无中止，其他数目到达停止
返回值	参考第 5 章函数返回码定义
说明	无终止循环设定行数
运行用时	**约 1ms**
支持版本	1.4.17

6.2.43 GLX8_2_SetBatchOneTimeDataHandler

格式	`int CPP_STDCALL GLX8_2_SetBatchOneTimeDataHandler(int lDeviceId, GLX8_2_BatchOneTimeCallBack CallFunc);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● CallFunc 参考回调函数 GLX8_2_BatchOneTimeCallBack
返回值	参考第 5 章函数返回码定义
说明	设置回调函数,建议获取数据后另外开启线程进行处理（获取数据模式:批处理一次回调一次）
运行用时	**约 1ms**
支持版本	1.4.17

6.2.44 GLX8_2_SetBatchOneTimeDataHandler_V2

格式	`int CPP_STDCALL GLX8_2_SetBatchOneTimeDataHandler_V2(int lDeviceId, std::function<void(const GLX8_2_STR_CALLBACK_INFO* info, const GLX8_2_Data DataObj)> fc);`
参数	● lDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● fc 泛型函数对象
返回值	参考第 5 章函数返回码定义
说明	设置回调函数,建议获取数据后另外开启线程进行处理（获取数据模式:批处理一次回调一次）
运行用时	**约 1ms**
支持版本	1.4.17

6.2.45 GLX8_2_GetBatchProfilePoint

格式	`int32_t * CPP_STDCALL GLX8_2_GetBatchProfilePoint(GLX8_2_Data DataIndex, int Head);`

© 2025 光子（深圳）精密 版权所有

参数	● DataIndex 回调函数返回的 DataObj 指针 ● Head 默认为 0

返回值	返回高度数据指针，单位是 0.01 微米， / 100000 = 毫米
	回调函数内使用
	`std::vector<int32_t> heightdatas;`
	`heightdatas.resize(info->BatchPoints * info->xPoints);`
	`int32_t* bdatas = GLX8_2_GetBatchProfilePoint(DataObj, 0);`
	`if (bdatas)`
	`{`
	`memcpy(heightdatas.data(), bdatas, heightdatas.size() * 4);`
	`}`

说明	批处理高度数据获取（获取数据模式:批处理一次回调一次）
运行用时	约 1ms
支持版本	1.4.17

6.2.46 GLX8_2_GetBatchProfilePoint16

格式	`uint16_t* CPP_STDCALL GLX8_2_GetBatchProfilePoint16(GLX8_2_Data DataIndex, int Head,double * res);`
参数	● DataIndex 回调函数返回的 DataObj 指针 ● Head 默认为 0 ● Res 分辨率

返回值	返回高度数据指针，单位是 0.01 微米， / 100000 = 毫米
	回调函数内使用
	`double res = 0.0f;`
	`uint16_t* pDeep16 = GLX8_2_GetBatchProfilePoint16(DataObj, 0, &res);`
	`if (pDeep16!=nullptr)`
	`{`
	`//16 位深度数据存在该指针中`
	`//转为真实值公式:(pDeep16[i]-32768)*res 单位：um`
	`}`

说明	批处理高度数据获取（获取数据模式:批处理一次回调一次）
运行用时	约 1ms
支持版本	1.4.17

© 2025 光子（深圳）精密 版权所有

6.2.47 GLX8_2_GetBatchIntensityPoint

格式	uint8_t * CPP_STDCALL GLX8_2_GetBatchIntensityPoint(GLX8_2_Data DataIndex, int Head);
参数	● DataIndex 回调函数返回的 DataObj 指针 ● Head 默认为 0

返回值	返回灰度数据指针
	回调函数内使用
	std::vector<uint8_t> grays;
	grays.resize(info->BatchPoints * info-x>Points);
	
	auto bdatas = GLX8_2_GetBatchIntensityPoint(DataObj, 0);
	if (bdatas)
	{
	memcpy(grays.data(), bdatas, grays.size());
	}

说明	批处理亮度获取（获取数据模式:批处理一次回调一次）
运行用时	约 1ms
支持版本	1.4.17

6.2.48 GLX8_2_GetBatchEncoderPoint

格式	uint32_t * CPP_STDCALL GLX8_2_GetBatchEncoderPoint(GLX8_2_Data DataIndex, int Head);
参数	● DataIndex 回调函数返回的 DataObj 指针 ● Head 默认为 0

返回值	返回编码器数据指针
	回调函数内使用
	std::vector<uint32_t> ens;
	ens.resize(info->BatchPoints);
	
	auto bdatas = GLX8_2_GetBatchEncoderPoint(DataObj, 0);
	if (bdatas)
	{
	memcpy(ens.data(), bdatas, ens * 4);
	}

说明	批处理编码器数据获取（获取数据模式:批处理一次回调一次）
运行用时	约 1ms
支持版本	1.4.17

© 2025 光子（深圳）精密 版权所有

6.2.49 GLX8_2_GetBatchTimes

格式	uint32_t CPP_STDCALL GLX8_2_GetBatchTimes(int lDeviceId, GLX8_2_Data DataObj);
参数	● LDeviceId 指定通过哪台通信设备进行通信，范围 0-255 ● GLX8_2_Data 数据对象
返回值	返回批处理次数
说明	批处理次数获取
运行用时	约 1ms
支持版本	1.4.17

6.2.50 GLX8_2_GetBatchTimestamps

格式	uint64_t* CPP_STDCALL GLX8_2_GetBatchTimestamps(GLX8_2_Data DataIndex, int Head);
参数	● DataIndex 回调函数返回的 DataObj 指针 ● Head 默认为 0

返回值	返回批处理时间戳
	回调函数内使用
	std::vector<uint64_t> tdatas;
	tdatas.resize(info->BatchPoints);
	uint64_t* tdata = GLX8_2_GetBatchTimestamps(DataObj, 0);
	if (tdata)
	{
	memcpy(tdatas.data(), tdata, tdatas.size() * 8);
	}

说明	批处理时间戳 单位 us（获取数据模式:批处理一次回调一次）
运行用时	约 1ms
支持版本	1.4.17

6.2.51 GLX8_2_ImageDataToPointCloud

格式	int CPP_STDCALL GLX8_2_ImageDataToPointCloud(int lDeviceId, L42_ImageData image_data, L42_CloudData cloud_data);
参数	● lDeviceId 设备 ID ● L42_ImageData 点云输入信息 ● L42_CloudData 点云输出结果

© 2025 光子（深圳）精密 版权所有

返回值	参考第 5 章函数返回码定义
说明	批处理时间戳 单位 us（获取数据模式:批处理一次回调一次）
运行用时	约 1ms
支持版本	1.6.35

7 软件开发接口函数调用顺序

7.1 设置获取参数

7.1.1 示例代码(C++)

```
int main()
{
    //设备 ip
    GLX8_2_ETHERNET_CONFIG config;
    config.abyIpAddress[0] = 192;
    config.abyIpAddress[1] = 168;
    config.abyIpAddress[2] = 1;
    config.abyIpAddress[3] = 10;
    //打开设备
     int ret = GLX8_2_EthernetOpen(0, &config);
    if (ret != 0) {
        std::cerr << "设备打开失败" << std::endl;
        return 0;
    }
    int depth = 0x01;
    int type = 0x10;
    int target[4] = { 0 };
    //批处理开关 0-关 1-开
    int iSwich= 1;
    //批处理点数 /图片高
    int height = 765;
    //触发模式 0-连续触发 2-编码器触发
    int TriMode = 2;
#pragma region 参数设置
    ret = GLX8_2_SetSetting(0, depth, type, 0, 0x03, target, (char*)&iSwich, 4);
    if (ret != 0)
    {
        iSwich= -1;
        std::cout << "批处理开关设置错误" << std::endl;

    }
```


© 2025 光子（深圳）精密 版权所有

```
32.      ret = GLX8_2_SetSetting(0, depth, type, 0, 0x0A, target, (char*)&height, 4);
33.      if (ret != 0)
34.      {
35.          height = -1;
36.          std::cout << "批处理点数设置错误" << std::endl;
37.
38.      }
39.      ret = GLX8_2_SetSetting(0, depth, type, 0, 0x01, target, (char*)&TriMode, 4);
40.      if (ret != 0)
41.      {
42.          TriMode = -1;
43.          std::cout << "触发模式设置错误" << std::endl;
44.
45.      }
46.      std::cout << "设置完成" << std::endl;
47.  #pragma endregion
48.  #pragma region 参数获取
49.      //复位设置参数
50.      iSwich = 0;
51.      height = 0;
52.      TriMode = 0;
53.      ret = GLX8_2_GetSetting(0, type, 0, 0x03, target, (char*)&iSwich, 4);
54.      if (ret != 0)
55.      {
56.          iSwich = -1;
57.          std::cout << "批处理开关获取错误" << std::endl;
58.
59.      }
60.      ret = GLX8_2_GetSetting(0, type, 0, 0x0A, target, (char*)&height, 4);
61.      if (ret != 0)
62.      {
63.          height = -1;
64.          std::cout << "批处理点数获取错误" << std::endl;
65.
66.      }
67.      ret = GLX8_2_GetSetting(0, type, 0, 0x01, target, (char*)&TriMode, 4);
68.      if (ret != 0)
69.      {
70.          TriMode = -1;
71.          std::cout << "触发模式获取错误" << std::endl;
72.
```


© 2025 光子（深圳）精密 版权所有

```
        }

        std::cout << "获取完成" << std::endl;
    #pragma endregion
        system("pause");
        return 0;
    }
```

7.1.2 示例代码(C#)

```
static void Main(string[] args)
{
    //设备 ip
    GLX8_2_ETHERNET_CONFIG config;
    config.abyIpAddress = new byte[4] {192,168,1,10 };
    const int DEVICE_ID = 0;
    //打开设备
    int ret = GLX8_2_EthernetOpen(DEVICE_ID,ref config);
    if (ret!=0)
    {
        Console.WriteLine("设备打开失败");
        return ;
    }
    int depth = 0x01;
    int type = 0x10;
    int iSwitch = 1;
    int height = 1009;
    int TriMode = 2;
    //开辟空间
    IntPtr data = Marshal.AllocHGlobal(4);
    #region 参数设置
    Marshal.WriteInt32(data, iSwitch);
    //批处理设置
    ret = GLX8_2_SetSetting(DEVICE_ID, depth, type, 0, 0x03, IntPtr.Zero, da
ta, 4);
    if (ret!=0)
    {
        iSwitch = -1;
        Console.WriteLine("批处理开关设置错误");
    }
    //批处理点数设置
    Marshal.WriteInt32(data, height);
    ret = GLX8_2_SetSetting(DEVICE_ID, depth, type, 0, 0x0A, IntPtr.Zero, da
ta, 4);
```


© 2025 光子（深圳）精密 版权所有

```
32.     if (ret!=0)
33.     {
34.         height = -1;
35.         Console.WriteLine("批处理点数设置错误");
36.     }
37.     //触发模式设置
38.     Marshal.WriteInt32(data,TriMode);
39.     ret = GLX8_2_SetSetting(DEVICE_ID, depth, type, 0, 0x01, IntPtr.Zero, da
    ta, 4);
40.     if (ret!=0)
41.     {
42.         TriMode = -1;
43.         Console.WriteLine("触发模式设置错误");
44.     }
45.     Console.WriteLine("设置完成");
46.     #endregion
47.     #region
48.     iSwitch=99;
49.     height = 0;
50.     TriMode = 99;
51.     //获取批处理开关设置
52.     ret = GLX8_2_GetSetting(DEVICE_ID,  type, 0, 0x03, IntPtr.Zero, data, 4);

53.     if (ret != 0)
54.     {
55.         iSwitch = -1;
56.         Console.WriteLine("批处理开关获取错误");
57.     }
58.     iSwitch=Marshal.ReadInt32(data);
59.     //获取批处理点数设置
60.     ret = GLX8_2_GetSetting(DEVICE_ID,  type, 0, 0x0A, IntPtr.Zero, data, 4);

61.     if (ret != 0)
62.     {
63.         height = -1;
64.         Console.WriteLine("批处理点数获取错误");
65.     }
66.     height=Marshal.ReadInt32(data);
67.     //获取触发模式设置
68.     ret = GLX8_2_GetSetting(DEVICE_ID,  type, 0, 0x01, IntPtr.Zero, data, 4);

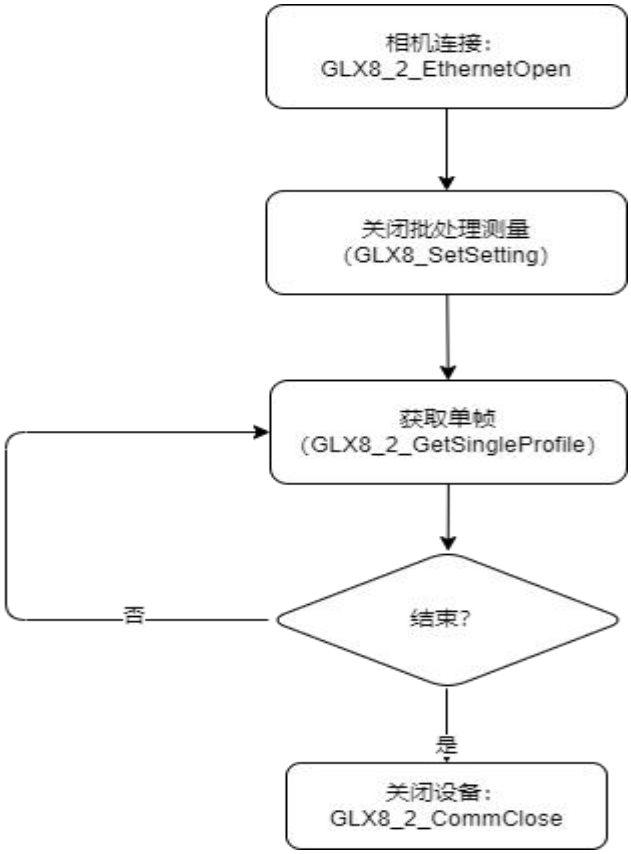
69.     if (ret != 0)
70.     {
```


© 2025 光子（深圳）精密 版权所有

```
71.         TriMode = -1;
72.         Console.WriteLine("触发模式获取错误");
73.     }
74.         TriMode =Marshal.ReadInt32(data);
75.     #endregion
76.     Console.ReadLine();
77.
78. }
```

7.2 单轮廓测量

7.2.1 调用顺序



7.2.2 示例代码(C++)

```
1. /*
2.     获取单轮廓方式需要关闭批处理
3.     参数设置可以在上位机中设置也可以在 SDK 中设置
4. */
5. int main() {
6.     //相机 ip
```


© 2025 光子（深圳）精密 版权所有

```
    GLX8_2_ETHERNET_CONFIG cnf;
    cnf.abyIpAddress[0] = 192;
    cnf.abyIpAddress[1] = 168;
    cnf.abyIpAddress[2] = 1;
    cnf.abyIpAddress[3] = 10;
    const int DEVICE_ID = 0;
    //打开设备
    int ret = GLX8_2_EthernetOpen(DEVICE_ID, &cnf);
    if (ret != 0) {
        return 0;
    }
    GLX8_2_ModelInfo info;
    ret = GLX8_2_GetModelInfos(DEVICE_ID,&info);
    //7000Series 3200
    int outline[4096] = {0};
    unsigned int enc = 0;
    int sunLines = 0;
    while (true) {
        //获取单轮廓
        ret=GLX8_2_GetSingleProfile(DEVICE_ID, outline, &enc);
        std::this_thread::sleep_for(std::chrono::microseconds(50));
        if (ret!=0)
        {
            std::cout << "轮廓获取异常" << std::endl;
            continue;
        }
        sunLines++;
        std::cout << "当前轮廓数: " << sunLines << std::endl;
    }
    //关闭设备
    GLX8_2_CommClose(DEVICE_ID);
    return 0;
}
```

7.2.3 示例代码(C#)

```
/*
    获取单轮廓方式需要关闭批处理
    参数设置可以在上位机中设置也可以在 SDK 中设置
*/
 static void Main(string[] args)
 {
    //相机 Ip
    GLX8_2_ETHERNET_CONFIG config = new GLX8_2_ETHERNET_CONFIG();
```

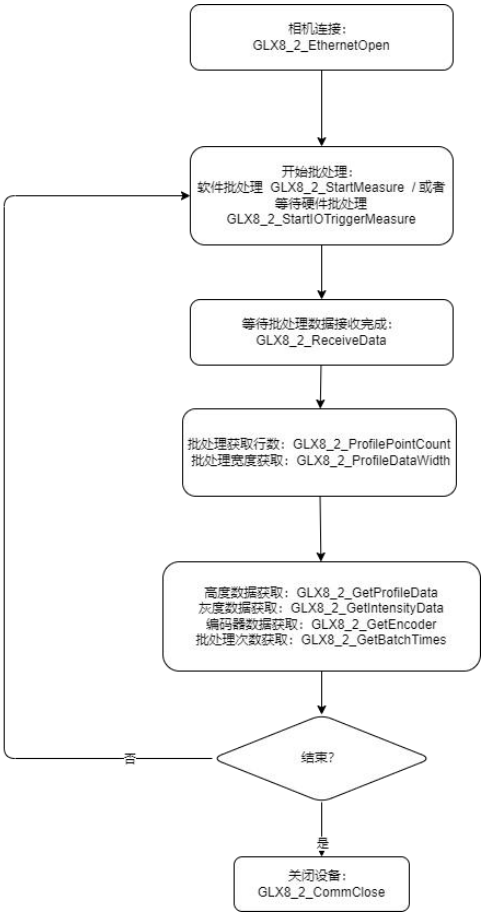

© 2025 光子（深圳）精密 版权所有

```
9.      config.abyIpAddress = new byte[4] {192,168,1,10 };
10.     const int DEVICE_ID = 0;
11.    //打开设备
12.    int ret=GLX8_2_EthernetOpen(DEVICE_ID,ref config);
13.   //获取一些相机信息
14.     GLX8_2_ModelInfo gLX8_2_ModelInfo;
15.     ret = GLX8_2_GetModelInfos(DEVICE_ID,out gLX8_2_ModelInfo);
16.     //预留空间存储
17.     int[] outLine = new int[gLX8_2_ModelInfo.ProfileDataWidth];
18.    IntPtr pOutLine = Marshal.AllocHGlobal(outLine.Length*sizeof(int));
19.      uint enc = 0;
20.     int sumLines = 0;
21.     while (true)
22.     {
23.          ret = GLX8_2_GetSingleProfile(DEVICE_ID, pOutLine, out enc);
24.
25.         if (ret!=0)
26.          {
27.             Console.WriteLine($"{sumLines}轮廓获取异常");
28.             continue;
29.          }
30.        //这里分辨率是 10nm
31.         Marshal.Copy(pOutLine, outLine, 0, outLine.Length);
32.        sumLines++;
33.        Console.WriteLine($"当前轮廓数:{sumLines}");
34.     }
35.
36.       Console.ReadLine();
37. }
```


© 2025 光子（深圳）精密 版权所有

7.3 阻塞方式获取数据

7.3.1 调用顺序



7.3.2 示例代码(C++)

```
/*
    阻塞方式需要打开批处理
    参数设置可以在上位机中设置也可以在 SDK 中设置
*/
int main()
{
    //相机 IP 地址
    GLX8_2_ETHERNET_CONFIG GLEthernetConFig;
    GLEthernetConFig.abyIpAddress[0] = 192;
     GLEthernetConFig.abyIpAddress[1] = 168;
     GLEthernetConFig.abyIpAddress[2] = 1;
     GLEthernetConFig.abyIpAddress[3] = 10;
     //设备 ID
     const int DEVICE_ID = 0;
     //连接相机
```


© 2025 光子（深圳）精密 版权所有

```
    int ret=GLX8_2_EthernetOpen(DEVICE_ID, &GLEthernetConFig);
    //开始批处理---------IO 触发时调用函数 GLX8_2_StartIOTriggerMeasure
    ret=GLX8_2_StartMeasure(DEVICE_ID, 20000);
    //等待数据接收
    GLX8_2_Data DataObject = NULL;
    std::cout << "数据收取中.." << std::endl;
    ret=GLX8_2_ReceiveData(0, DataObject);
    std::cout << "收取完成" << std::endl;
    //获取批处理行数
    int BatchPoint = GLX8_2_ProfilePointCount(DEVICE_ID, DataObject);
    //获取轮廓宽度
    int m_DataWidth = GLX8_2_ProfileDataWidth(DEVICE_ID, DataObject);
    //定义高度数据缓存    深度图数据存在改指针中
    int* HeightData = new int[BatchPoint * m_DataWidth];
    //获取高度数据;
    ret=GLX8_2_GetProfileData(DEVICE_ID, DataObject, HeightData);

    //定义灰度数据缓存     灰度图数据存在该指针中
    unsigned char* grayData = new unsigned char[BatchPoint * m_DataWidth];
    //获取灰度数据
    ret=GLX8_2_GetIntensityData(DEVICE_ID, DataObject, grayData);
    //定义编码器数据缓存
    unsigned int* Encoder = new unsigned int[BatchPoint];
    //获取编码器数据
    ret=GLX8_2_GetEncoder(DEVICE_ID, DataObject, Encoder);
    //停止批处理
    ret=GLX8_2_StopMeasure(DEVICE_ID);
    //关闭设备
    GLX8_2_CommClose(DEVICE_ID);
    //内存释放
    delete[] HeightData;
    delete[] grayData;
    delete[] Encoder;
    system("pause");
    return 0;
}
```

7.3.3 示例代码(C#)

```
/*
  阻塞方式需要打开批处理
  参数设置可以在上位机中设置也可以在 SDK 中设置
*/
static void Main(string[] args) {
```


© 2025 光子（深圳）精密 版权所有

```
    // 相机 IP 地址
    GLX8_2_ETHERNET_CONFIG config = new GLX8_2_ETHERNET_CONFIG();
    config.abyIpAddress = new byte[4] {  192,  168,   1,  10  };
    // 设备 ID
      const int DEVICE_ID = 0;
      // 连接相机
      int ret = GLX8_2_EthernetOpen(0, ref config);
      // 开始批处理---------IO 触发时调用函数 GLX8_2_StartIOTriggerMeasure
      ret = GLX8_2_StartMeasure(DEVICE_ID, 20000);
      GLX8_2_Data data;
      // 等待数据接收
      Console.WriteLine("数据收取中..");
      ret = GLX8_2_ReceiveData(0, out data);
      Console.WriteLine("收取完成");
      // 获取批处理行数
      int BatchPoint = GLX8_2_ProfilePointCount(DEVICE_ID, IntPtr.Zero);
      // 获取轮廓宽度
      int m_DataWidth = GLX8_2_ProfileDataWidth(DEVICE_ID, IntPtr.Zero);
      // 定义高度数据缓存    深度图数据存在该数组中
      int[] heightData = new int[BatchPoint * m_DataWidth];
      IntPtr ptrHei = Marshal.AllocHGlobal(heightData.Length * sizeof(int));
      // 定义灰度数据缓存     灰度图数据存在该指针中
      byte[] grayData = new byte[BatchPoint * m_DataWidth];
      IntPtr ptrGray = Marshal.AllocHGlobal(grayData.Length * sizeof(byte));
      // 编码器数据缓存
      byte[] encoderTemp = new byte[BatchPoint * 4];
      IntPtr ptrEnchoder = Marshal.AllocHGlobal(encoderTemp.Length * sizeof(ui
  nt));
      UInt32[] encoder = new UInt32[BatchPoint];
      try {
          // 获取高度数据
          ret = GLX8_2_GetProfileData(DEVICE_ID, IntPtr.Zero, ptrHei);
          Marshal.Copy(ptrHei, heightData, 0, heightData.Length);
          // 获取灰度数据
          ret = GLX8_2_GetIntensityData(DEVICE_ID, IntPtr.Zero, ptrGray);
          Marshal.Copy(ptrGray, grayData, 0, grayData.Length);
          // 获取编码器数据
          ret = GLX8_2_GetEncoder(DEVICE_ID, IntPtr.Zero, ptrEnchoder);
          Marshal.Copy(ptrEnchoder, encoderTemp, 0, encoderTemp.Length);
          for (int i = 0; i < encoderTemp.Length / 4; i++) {
              encoder[i] = BitConverter.ToUInt32(encoderTemp, i * 4);
          }

```

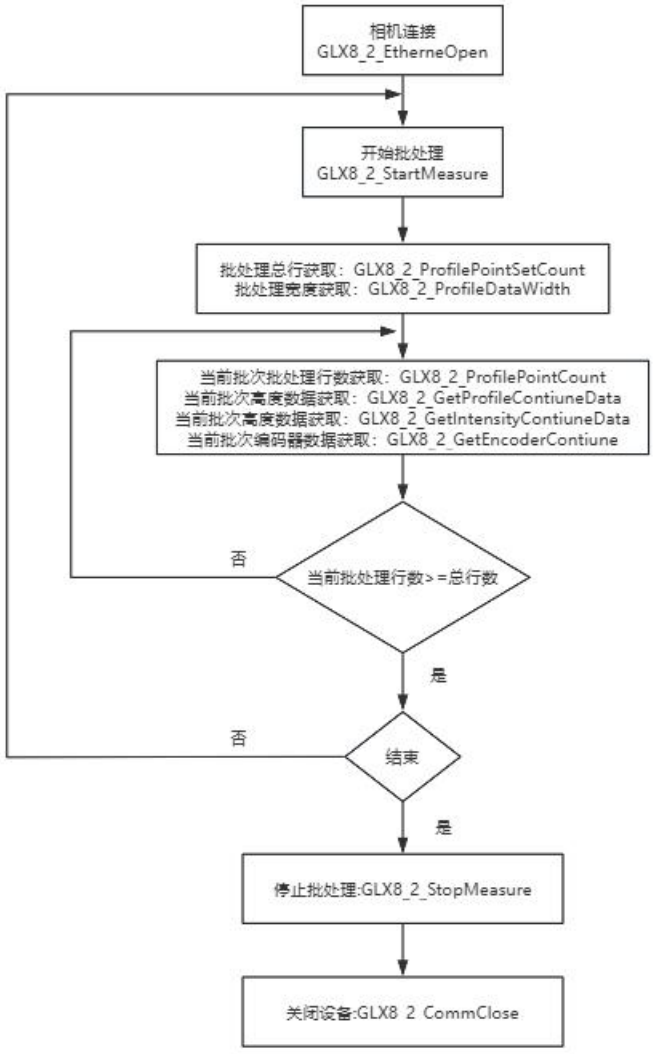

© 2025 光子（深圳）精密 版权所有

```
48.     } catch (Exception) {
49.         Marshal.FreeHGlobal(ptrHei);
50.         Console.WriteLine("数据拷贝失败！");
51.         return;
52.     }
53.
54.     // 停止批处理
55.     ret = GLX8_2_StopMeasure(DEVICE_ID);
56.     // 关闭设备
57.     GLX8_2_CommClose(DEVICE_ID);
58.     //清理内存
59.     if (ptrHei != IntPtr.Zero)
60.         Marshal.FreeHGlobal(ptrHei);
61.     if (ptrGray != IntPtr.Zero)
62.         Marshal.FreeHGlobal(ptrGray);
63.     if (ptrEnchoder != IntPtr.Zero)
64.         Marshal.FreeHGlobal(ptrEnchoder);
65.     Console.ReadKey();
66.     return;
67. }
```

7.4 非阻塞方式有限循环获取数据

7.4.1 调用顺序

© 2025 光子（深圳）精密 版权所有



7.4.2 示例代码(C++)

```
/*
    连续获取模式需要打开批处理  行数为实际设置的行数
    参数设置可以在上位机中设置也可以在 SDK 中设置
*/
int main()
{
    // 相机 IP
    GLX8_2_ETHERNET_CONFIG cnf;
    const int DEVICE_ID = 0;
    cnf.abyIpAddress[0] = 192;
    cnf.abyIpAddress[1] = 168;
    cnf.abyIpAddress[2] = 1;
    cnf.abyIpAddress[3] = 10;
```


© 2025 光子（深圳）精密 版权所有

```
    int ret = GLX8_2_EthernetOpen(DEVICE_ID, &cnf);
    GLX8_2_Data DataObject = NULL;
    // 开始批处理---IO 触发时用函数 GLX8_2_StartIOTriggerMeasure
    GLX8_2_StartMeasure(DEVICE_ID, 2000);
    // 批处理行数获取
    int BatchPoint = GLX8_2_ProfilePointSetCount(DEVICE_ID, DataObject);
    // 获取轮廓宽度
    int m_DataWidth = GLX8_2_ProfileDataWidth(DEVICE_ID, DataObject);
    // 上一次接收到的批处理数据
    int lastBatchPoint_CurNo = 0;
    // 高度数据缓存
    int* HeightData = new int[BatchPoint * m_DataWidth];
    // 灰度数据缓存
    unsigned char* grayData = new unsigned char[BatchPoint * m_DataWidth];
    // 编码器数据缓存
    unsigned int* Encoder = new unsigned int[BatchPoint];
    do {
        // 获取当前批次的批处理行数编号
        int BatchPoint_CurNo = GLX8_2_ProfilePointCount(DEVICE_ID, DataObject);

        // 当前批次批处理行数
        int m_curBatchPoint = BatchPoint_CurNo - lastBatchPoint_CurNo;
        // 获取当前批次高度数据
        GLX8_2_GetProfileContiuneData(DEVICE_ID, DataObject, &HeightData[lastB
atchPoint_CurNo * m_DataWidth], m_curBatchPoint);
        // 获取当前批次灰度数据
        GLX8_2_GetIntensityContiuneData(DEVICE_ID, DataObject, &grayData[lastB
atchPoint_CurNo * m_DataWidth], m_curBatchPoint);
        // 获取当前批次编码数据
        GLX8_2_GetEncoderContiune(DEVICE_ID, DataObject, &Encoder[lastBatchPoi
nt_CurNo], m_curBatchPoint);
        lastBatchPoint_CurNo = BatchPoint_CurNo;
        std::cout << "batch num: " << BatchPoint_CurNo << std::endl;
        std::this_thread::sleep_for(std::chrono::microseconds(50));

    } while (lastBatchPoint_CurNo < BatchPoint);

    GLX8_2_StopMeasure(DEVICE_ID); // 关闭设备
    GLX8_2_CommClose(DEVICE_ID);
    // 内存释放
    delete[] HeightData;
    delete[] grayData;
    delete[] Encoder; // 停止批处理
```


© 2025 光子（深圳）精密 版权所有

```
    system("Pause");
    return 0;

}
```

7.4.3 示例代码(C#)

```
/*
    连续获取模式需要打开批处理 行数为实际设置的行数
    参数设置可以在上位机中设置也可以在 SDK 中设置
*/
static void Main(string[] args) {
    Console.WriteLine(Environment.CurrentDirectory);
    // 配置设备以太网连接信息
    GLX8_2_ETHERNET_CONFIG config = new GLX8_2_ETHERNET_CONFIG();
    config.abyIpAddress = new byte[4] {  192,  168,  1,  10  };
    const int DEVICE_ID = 0;
    // 打开设备连接（设备号 0）
    int ret = GLX8_2_EthernetOpen(0, ref config);

    // 开始批处理---IO 触发时间用函数 GLX8_2_StartIOTriggerMeasure
    ret = GLX8_2_StartMeasure(0, 2000);

    // 批处理行数获取
    int BatchPoint = GLX8_2_ProfilePointSetCount(DEVICE_ID, IntPtr.Zero);
    // 获取轮廓宽度
    int dataWidth = GLX8_2_ProfileDataWidth(DEVICE_ID, IntPtr.Zero);
    // 上一次接收到的批处理数据
    int lastBatchPoint_CurNo = 0;
    // 高度数据缓存
    int[] heightData = new int[BatchPoint * dataWidth];
    IntPtr ptrHeight = Marshal.AllocHGlobal(heightData.Length * sizeof(int));

    // 灰度数据缓存
    byte[] grayData = new byte[BatchPoint * dataWidth];
    IntPtr ptrGray = Marshal.AllocHGlobal(grayData.Length * sizeof(byte));
    // 编码器数据缓存
    byte[] encoderData = new byte[BatchPoint * 4];
    IntPtr ptrEncoder = Marshal.AllocHGlobal(encoderData.Length * sizeof(byte))
;

    do {
        // 获取当前批次的批处理行数编号
```


© 2025 光子（深圳）精密 版权所有

```
        int BatchPoint_CurNo = GLX8_2_ProfilePointCount(DEVICE_ID, IntPtr.Zero);
        if (BatchPoint_CurNo < 0) {
            continue;
        }
        // 当前批次批处理行数
        uint m_curBatchPoint = (uint)(BatchPoint_CurNo - lastBatchPoint_CurNo);

        // 获取当前批次高度数据
        IntPtr heightOffset = IntPtr.Add(ptrHeight, lastBatchPoint_CurNo * dataWidth * sizeof(int));
        GLX8_2_GetProfileContiuneData(DEVICE_ID, IntPtr.Zero, heightOffset, m_curBatchPoint);
        // 获取当前批次灰度数据
        IntPtr grayOffset = IntPtr.Add(ptrGray, lastBatchPoint_CurNo * dataWidth * sizeof(byte));
        GLX8_2_GetIntensityContiuneData(DEVICE_ID, IntPtr.Zero, grayOffset, m_curBatchPoint);
        // 获取当前批次编码数据
        IntPtr encoderOffset = IntPtr.Add(ptrEncoder, lastBatchPoint_CurNo * sizeof(byte) * 4);
        GLX8_2_GetEncoderContiune(DEVICE_ID, IntPtr.Zero, encoderOffset, m_curBatchPoint);
        lastBatchPoint_CurNo = BatchPoint_CurNo;
        Console.WriteLine($ "batchnum:{BatchPoint_CurNo} ");
        Thread.Sleep(500);
    } while (lastBatchPoint_CurNo < BatchPoint);
    //拷贝数据到数组中
    Marshal.Copy(ptrHeight, heightData, 0, BatchPoint * dataWidth);
    Marshal.Copy(ptrGray, grayData, 0, BatchPoint * dataWidth);
    Marshal.Copy(ptrEncoder, encoderData, 0, BatchPoint * 4);
    //停止采集
    GLX8_2_StopMeasure(DEVICE_ID);
    // 关闭设备
    GLX8_2_CommClose(DEVICE_ID);
    //清理内存
    if (ptrHeight != IntPtr.Zero)
        Marshal.FreeHGlobal(ptrHeight);
    if (ptrGray != IntPtr.Zero)
        Marshal.FreeHGlobal(ptrGray);
    if (ptrEncoder != IntPtr.Zero)
        Marshal.FreeHGlobal(ptrEncoder);

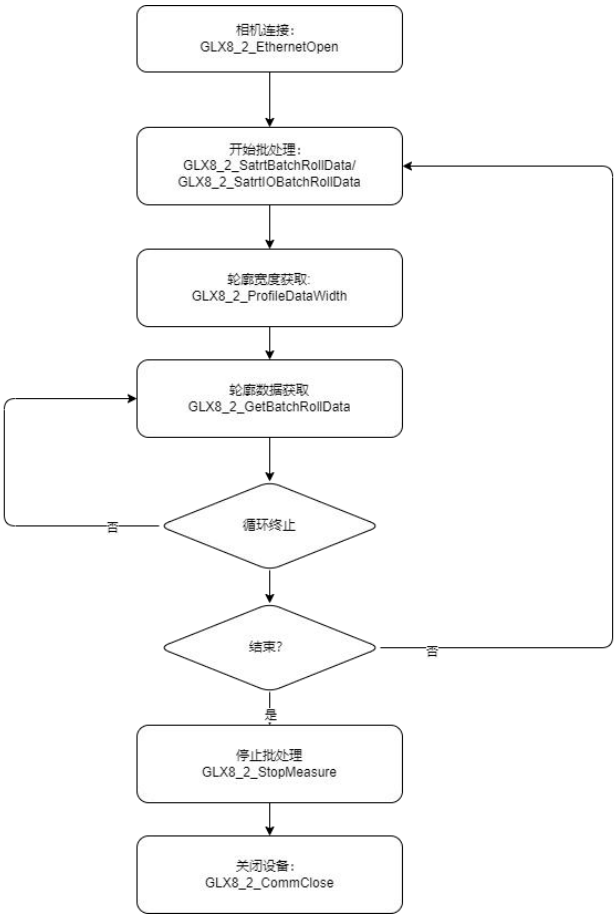
```


© 2025 光子（深圳）精密 版权所有

```
}
```

7.5 非阻塞方式无限循环

7.5.1 调用顺序



7.5.2 示例代码(C++)

```cpp
/*
    无限循环方式需要打开批处理
    参数设置可以在上位机中设置也可以在 SDK 中设置
*/
int main()
{
    const int DEVICE_ID = 0;
    // 定义一个设备配置结构体
    GLX8_2_ETHERNET_CONFIG config;
    // 设置设备的 IP 地址为 192.168.1.11
    config.abyIpAddress[0] = 192;
    config.abyIpAddress[1] = 168;
    config.abyIpAddress[2] = 1;
```


© 2025 光子（深圳）精密 版权所有

```
    config.abyIpAddress[3] = 10;
    // 调用函数 GLX8_2_EthernetOpen 打开设备，设备编号为 0，使用上面的配置
    int ret = GLX8_2_EthernetOpen(DEVICE_ID, &config);
    //开始批处理---------IO 触发时调用函数 GLX8_2_StartIOBatchRollData
    ret = GLX8_2_StartBatchRollData(DEVICE_ID);
    // 定义一个变量，用于存储数据的宽度
    GLX8_2_Data data= NULL;
    // 获取数据的宽度，设备编号为 DEVICE_ID，数据对象为 data
    int iDataWidth = GLX8_2_ProfileDataWidth(DEVICE_ID, data);
    //接收的行数
    int sumLines = 10000;
    // 动态分配内存，用于存储高度数据，大小为 sumLines * 数据宽度
    int* HeightData = new int[sumLines * iDataWidth];
    // 动态分配内存，用于存储灰度数据，大小为 sumLines * 数据宽度
    unsigned char* GrayData = new unsigned char[sumLines * iDataWidth];
    // 动态分配内存，用于存储帧丢失信息，大小为 sumLines
    unsigned int* FrameLoss = new unsigned int[sumLines];
    // 动态分配内存，用于存储帧 ID，大小为 sumLines
    long long* FrameId = new long long[sumLines];
    // 动态分配内存，用于存储编码器信息，大小为 sumLines
    unsigned int* Encoder = new unsigned int[sumLines];
    // 定义变量，用于记录溢出开始的帧 ID
    long long OverFlowstartlD = 0;
    //定义变量，用于记录帧丢失的 ID
    int FrameLosslD = 0;
    // 定义变量，用于记录编码器的 ID
    int EncoderldD =0;
    //定义变量，用于记录获取总行数
    int BatchPointCurNo = 0;
    // 开始循环，直到行数达到要求
    do
    {
        int remainingPoints = sumLines - BatchPointCurNo;
        int batchSize =std::min(500, remainingPoints);
        if (remainingPoints <= 0) break;
        // 调用函数 GLX8_2_GetBatchRollData 获取批量滚动数据，设备编号为 0，数据对象为 data1，高度数据存储在 HeightData，灰度数据存储在 GrayData，编码器信息存储在 Encoder，帧 ID 存储在 FrameId，帧丢失信息存储在 FrameLoss，最多获取 500 个数据点
        int m_curBatchPoint = GLX8_2_GetBatchRollData(DEVICE_ID, data, &HeightData[BatchPointCurNo*iDataWidth], &GrayData[BatchPointCurNo*iDataWidth], &Encoder[BatchPointCurNo], &FrameId[BatchPointCurNo], &FrameLoss[BatchPointCurNo], batchSize);
```


© 2025 光子（深圳）精密 版权所有

```
        // 输出当前批次的数据点数量
        std::cout<< "batchNum:" << m_curBatchPoint << std::endl;
        // 如果当前批次没有数据点，跳过本次循环
        if (0>= m_curBatchPoint)
        {
            continue;
        }
        // 获取当前批次最后一个数据点的索引
        int TmpID = m_curBatchPoint - 1;
        // 更新溢出开始的帧 ID
        OverFlowstartlD = FrameId[TmpID];
        // 更新帧丢失的 ID
        FrameLosslD = FrameLoss[TmpID];
        // 更新编码器的 ID
        EncoderldD = Encoder[TmpID];
        // 累加当前批次的数据点数量
        BatchPointCurNo += m_curBatchPoint;

        // 休眠 50 毫秒
        std::this_thread::sleep_for(std::chrono::milliseconds(50));

    }while(BatchPointCurNo<sumLines);

    GLX8_2_StopMeasure(DEVICE_ID);
    GLX8_2_CommClose(DEVICE_ID);
   // 释放动态分配的内存
    delete[] HeightData;
    delete[] GrayData;
    delete[] FrameLoss;
    delete[] FrameId;
    delete[] Encoder;
    return 0;

}
```

7.5.3 示例代码(C#)

```
/*无限循环方式需要打开批处理
   参数设置可以在上位机中设置也可以在 SDK 中设置
*/
static void Main(string[] args)
{
    //相机 IP
    const int DEVICE_ID = 0;
```


© 2025 光子（深圳）精密 版权所有

```
    GLX8_2_ETHERNET_CONFIG config = new GLX8_2_ETHERNET_CONFIG();
    config.abyIpAddress=new byte[4]{ 192,168,1,12};
      int ret = GLX8_2_EthernetOpen(DEVICE_ID,ref config);
      if (ret!=0)
      {
          Console.WriteLine("连接错误");
          return;
      }
      //开始批处理
      ret = GLX8_2_StartBatchRollData(DEVICE_ID);
      int iDataWidth = GLX8_2_ProfileDataWidth(DEVICE_ID, IntPtr.Zero);
      //总行号
      int sumLines = 5000;
      //存储轮廓图
      int[] heightData = new int[iDataWidth * sumLines];
      //存储亮度图
      byte[] grayData = new byte[iDataWidth * sumLines];
      //存储丢失帧
      byte[] frameLoseData = new byte[sumLines* sizeof(uint)];
      // 存储帧 ID
      byte[]frameldData = new byte[sumLines* sizeof(long)];
      // 存储编码器信息
      byte[] enCoderData = new byte[sumLines * sizeof(uint)];
      //存储轮廓图指针
      IntPtr heightDataPtr = Marshal.AllocHGlobal(sizeof(int) * iDataWidth * s
  umLines);
      //存储亮度图指针
      IntPtr grayDataPtr = Marshal.AllocHGlobal(sizeof(byte) * iDataWidth * su
  mLines);
      //存储丢失帧信息指针
      IntPtr frameLossPtr = Marshal.AllocHGlobal(sizeof(uint) * sumLines);
      //存储帧 ID 指针
      IntPtr frameIdPtr = Marshal.AllocHGlobal(sizeof(long) * sumLines);
      //存储编码器信息指针
      IntPtr encoderPtr = Marshal.AllocHGlobal(sizeof(uint) * sumLines);

      // 定义变量，用于记录溢出开始的帧 ID
      long overFlowStartId = 0;
      // 定义变量，用于记录帧丢失的 ID
      int frameLossId = 0;
      // 定义变量，用于记录编码器的 ID
      int encoderId = 0;
      // 定义变量，用于记录获取总行数
```


© 2025 光子（深圳）精密 版权所有

```
49.     int batchPointCurNo = 0;
50.     // 开始循环，直到 bStop 为 false
51.     do
52.     {
53.         int remainingPoints = sumLines - batchPointCurNo;
54.         int batchSize = Math.Min(500, remainingPoints);
55.
56.         // 检查是否还有数据需要读取
57.         if (remainingPoints <= 0)
58.             break;
59.         //计算偏移量
60.         IntPtr heightOffset = IntPtr.Add(heightDataPtr, batchPointCurNo * iDataWidth*sizeof(int));
61.         IntPtr grayOffset = IntPtr.Add(grayDataPtr, batchPointCurNo * iDataWidth*sizeof(byte));
62.         IntPtr frameLossOffset = IntPtr.Add(frameLossPtr, batchPointCurNo*sizeof(uint));
63.         IntPtr frameIdOffset = IntPtr.Add(frameIdPtr, batchPointCurNo*sizeof(long));
64.         IntPtr encoderOffset = IntPtr.Add(encoderPtr, batchPointCurNo*sizeof(uint));
65.
66.         // 获取批量滚动数据
67.         int currentBatchPoint = GLX8_2_GetBatchRollData(DEVICE_ID, IntPtr.Zero, heightOffset, grayOffset, encoderOffset, frameIdOffset, frameLossOffset, batchSize);
68.         // 输出当前批次的数据点数量
69.         Console.WriteLine($"batch Num: {currentBatchPoint}");
70.         // 如果当前批次没有数据点，跳过本次循环
71.         if (currentBatchPoint <= 0)
72.         {
73.             continue;
74.         }
75.         // 获取当前批次最后一个数据点的索引
76.         int tmpId = currentBatchPoint - 1;
77.         // 累加当前批次的数据点数量
78.         batchPointCurNo += currentBatchPoint;
79.         // 休眠 50 毫秒
80.         Thread.Sleep(50);
81.
82.     } while (batchPointCurNo < sumLines);
83.     //数据送进托管内存
84.     Marshal.Copy(heightDataPtr,heightData,0,iDataWidth*sumLines);
```

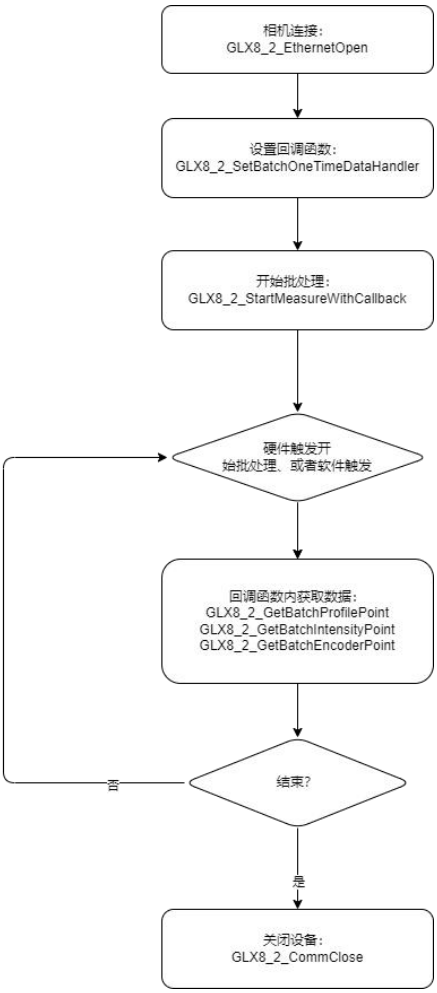

© 2025 光子（深圳）精密 版权所有

```
85.     Marshal.Copy(grayDataPtr, grayData, 0, iDataWidth * sumLines);
86.     Marshal.Copy(frameLossPtr, frameLoseData, 0,sumLines*sizeof(uint));
87.     Marshal.Copy(frameIdPtr, frameldData, 0, sumLines*sizeof(long));
88.     Marshal.Copy(encoderPtr, enCoderData, 0, sumLines*sizeof(uint));
89.
90.     GLX8_2_StopMeasure(DEVICE_ID);
91.     GLX8_2_CommClose(DEVICE_ID);
92.
93.     // 释放开辟的内存
94.     if (heightDataPtr != IntPtr.Zero)
95.         Marshal.FreeHGlobal(heightDataPtr);
96.     if (grayDataPtr != IntPtr.Zero)
97.         Marshal.FreeHGlobal(grayDataPtr);
98.     if (frameLossPtr != IntPtr.Zero)
99.         Marshal.FreeHGlobal(frameLossPtr);
100.    if (frameIdPtr != IntPtr.Zero)
101.        Marshal.FreeHGlobal(frameIdPtr);
102.    if (encoderPtr != IntPtr.Zero)
103.        Marshal.FreeHGlobal(encoderPtr);
104.
105.}
```

7.6 回调方式

7.6.1 调用顺序

© 2025 光子（深圳）精密 版权所有



7.6.2 示例代码(C++)

```
/*
    回调方式需要打开批处理 行数为实际设置的行数
    参数设置可以在上位机中设置也可以在 SDK 中设置
*/
 //把 z 数据转成 mm /100000 无效数据-999900000
void MyCallback(const GLX8_2_STR_CALLBACK_INFO* info,
    const GLX8_2_Data DataObj) {
    std::cout << std::endl;
    std::cout << "DataCallBack ------- DeviceID = " << DataObj->lDeviceId << std::endl;
    std::cout << "  xPoints      = " << info->xPoints << std::endl; // x 方向的数据点数
     std::cout << "  BatchPoints   = " << info->BatchPoints << std::endl; // 每批数据点数（y 方向）
     std::cout << "  BatchTimes    = " << info->BatchTimes  << std::endl; // 当前批次数量
```


© 2025 光子（深圳）精密 版权所有

```
12.      std::cout << "  xPixth       = " << info->xPixth << std::endl; // x 方向点间距
13.      std::cout << "  startEncoder = " << info->startEncoder << std::endl; // 开始编码器位置
14.      std::cout << "  HeadNumber   = " << info->HeadNumber << std::endl; // 相机头数量
15.      std::cout << "  returnStatus = " << info->returnStatus << std::endl; // 返回状态（0 表示正常）
16.      std::cout << "  EXTROI_EN    = " << info->EXTROI_EN << std::endl; // 扩展 ROI 状态（0 正常，1 表示行数不一致）
17.      std::cout << std::endl;
18.      // 如果采集状态异常则返回
19.      if (info->returnStatus != 0)
20.          return;
21.      // 获取深度图数据（Z 方向）
22.      int* pDeep = GLX8_2_GetBatchProfilePoint(DataObj, 0);
23.      if (pDeep != nullptr) {
24.          //深度数据存在该指针中
25.
26.      }
27.      //可选获取 16 位深度数据
28.      double factor=0.0f;
29.      int* pDeep16 = GLX8_2_GetBatchProfilePoint16(DataObj, 0,&factor);
30.      if (pDeep != nullptr) {
31.          //16 深度数据存在该指针中
32.          //转为真实值公式:(pDeep16[i]-32768)*res  单位：um
33.      }
34.
35.      // 获取灰度图数据
36.      uint8_t* pGray = GLX8_2_GetBatchIntensityPoint(DataObj, 0);
37.      if (pGray != nullptr) {
38.          //灰度数据存在该指针中
39.
40.      }
41.
42.  }
43.  int main()
44.  {
45.      // 配置设备以太网连接信息
46.      GLX8_2_ETHERNET_CONFIG config;
47.      config.abyIpAddress[0] = 192;
48.      config.abyIpAddress[1] = 168;
49.      config.abyIpAddress[2] = 1;
```


© 2025 光子（深圳）精密 版权所有

```
50.     config.abyIpAddress[3] = 10;
51.     // 打开设备连接（设备号 0）
52.     int ret = GLX8_2_EthernetOpen(0, &config);
53.     int depth = 0x01;
54.     int type = 0x10;
55.     int height = 1009;//这里设置点数
56.     int target[4] = { 0 };
57.     //批处理开关 0-关 1-开
58.     int iSwich = 1;
59. #pragma region 参数设置
60.     ret = GLX8_2_SetSetting(0, depth, type, 0, 0x03, target, (char*)&iSwich,
    4);
61.     if (ret != 0)
62.     {
63.         iSwich = -1;
64.         std::cout << "批处理开关设置错误" << std::endl;
65.
66.     }
67.     ret = GLX8_2_SetSetting(0, depth, type, 0, 0x0A, target, (char*)&height,
    4);
68.     if (ret != 0)
69.     {
70.         height = -1;
71.         std::cout << "批处理点数设置错误" << std::endl;
72.     }
73.     // 设置批处理数据回调函数
74.     ret = GLX8_2_SetBatchOneTimeDataHandler(0, MyCallback);
75.     // 获取设备型号信息（可选）
76.     GLX8_2_ModelInfo minfo;
77.     ret = GLX8_2_GetModelInfos(0, &minfo);
78.     // 开始批处理---IO 触发用函数 GLX8_2_StartIOTriggerMeasure
79.     ret = GLX8_2_StartMeasure(0, 2000);
80.     //第二个参数:为 0  调用一次触发一次设定的行数  io 触发只需要调用一次有 io 信号就
    触发
81.             //           为 1  只接收 io 信号，无法软触发
82.     //ret = GLX8_2_StartMeasureWithCallback(0, 1);
83.     std::cout << "回调中。。。。" << std::endl;
84.     system("pause");
85.     return 0;
86. }
```


© 2025 光子（深圳）精密 版权所有

7.6.3 示例代码(C#)

```
/*
    回调方式需要打开批处理 行数为实际设置的行数
    参数设置可以在上位机中设置也可以在 SDK 中设置
*/
//把 z 数据转成 mm /100000 无效数据-999900000

public static void MyCallback(IntPtr info, IntPtr DataObj)
{
    // 将 IntPtr 转换为结构体
        GLX8_2_STR_CALLBACK_INFO callbackInfo = Marshal.PtrToStructure<GLX8_2_STR_CALLBACK_INFO>(info);
    GLX8_2_Data dataObj = Marshal.PtrToStructure<GLX8_2_Data>(DataObj);
    Console.WriteLine();
    Console.WriteLine($"DataCallBack ------- DeviceID = {dataObj.lDeviceId}");

    Console.WriteLine($"  xPoints       = {callbackInfo.xPoints}"); // x 方向的数据点数
    Console.WriteLine($"  BatchPoints   = {callbackInfo.BatchPoints}"); // 每批数据点数（y 方向）
    Console.WriteLine($"  BatchTimes    = {callbackInfo.BatchTimes}"); // 当前批次数量
    Console.WriteLine($"  xPixth        = {callbackInfo.xPixth}"); // x 方向点间距  um
    Console.WriteLine($"  startEncoder = {callbackInfo.startEncoder}"); // 开始编码器位置
    Console.WriteLine($"  HeadNumber    = {callbackInfo.HeadNumber}"); // 相机头数量
    Console.WriteLine($"  returnStatus = {callbackInfo.returnStatus}"); // 返回状态（0 表示正常）

    Console.WriteLine();
    IntPtr pDeep = GLX8_2_GetBatchProfilePoint(ref dataObj, 0);
    IntPtr pGray = GLX8_2_GetBatchIntensityPoint(ref dataObj, 0);
    // 如果采集状态异常则返回
    if (callbackInfo.returnStatus != 0|| pDeep==IntPtr.Zero||pGray==IntPtr.Zero)
        return;
    int[] deepData = new int[callbackInfo.BatchPoints*callbackInfo.xPoints];
    // 获取深度图数据（Z 方向）
    if (pDeep != IntPtr.Zero)
    {
```


© 2025 光子（深圳）精密 版权所有

```
33.         //深度数据存在该指针中
34.         Marshal.Copy(pDeep,deepData,0,deepData.Length);
35.     }
36.    //根据需求可获取 16 位精度深度图
37.    double factor;
38.  IntPtr pDeep16 = GLX8_2_GetBatchProfilePoint16(ref dataObj, 0, out factor);
39.  if (pDeep16 != IntPtr.Zero)
40.  {
41.       short[] tempArray = new short[callbackInfo.BatchPoints * callbackInfo.x
    Points];
42.       Marshal.Copy(pDeep, tempArray, 0, callbackInfo.BatchPoints * callbackIn
    fo.xPoints);
1.       // result16 中无效值是 0  转成真实值公式:(result16[i]-32768)*factor  单位:
    um
2.      ushort[] result16 = Array.ConvertAll(tempArray, s => (ushort)s);
3. }
43.
44.     byte[] grayData = new byte[callbackInfo.BatchPoints * callbackInfo.xPoints]
    ;
45.     // 获取灰度图数据
46.     if (pGray != IntPtr.Zero)
47.     {
48.         //灰度数据存在该指针中
49.         Marshal.Copy(pGray,grayData,0, callbackInfo.BatchPoints * callbackInfo.
    xPoints);
50.     }
51.
52.
53. }
54.
55. public static int Main()
56. {
57.     // 配置设备以太网连接信息
58.     GLX8_2_ETHERNET_CONFIG config = new GLX8_2_ETHERNET_CONFIG();
59.     config.abyIpAddress = new byte[4] { 192, 168, 1, 12 };
60.
61.     // 打开设备连接（设备号 0）
62.     int ret = GLX8_2_EthernetOpen(0, ref config);
63.     GLX8_2_BatchOneTimeCallBack callback = new GLX8_2_BatchOneTimeCallBack(MyC
    allback);
64.
65.     // 使用 GCHandle 固定委托内存
66.     GCHandle callbackHandle = GCHandle.Alloc(callback, GCHandleType.Normal);
```


© 2025 光子（深圳）精密 版权所有

```
    // 设置批处理数据回调函数
    ret = GLX8_2_SetBatchOneTimeDataHandler(0, callback);

    // 获取设备型号信息（可选）
    GLX8_2_ModelInfo minfo ;
    ret = GLX8_2_GetModelInfos(0, out minfo);

    // 开始批处理---IO 触发用函数 GLX8_2_StartIOTriggerMeasure   触发一次需要调用一次
    //ret = GLX8_2_StartMeasure(0, 0);
    //第二个参数:为 0  调用一次触发一次设定的行数  io 触发只需要调用一次有 io 信号就触发
    //           为 1  只接收 io 信号，无法软触发
    ret = GLX8_2_StartMeasureWithCallback(0, 1);
    callbackHandle.Free();
    Console.WriteLine("回调中。。。。");
    Console.ReadLine();
    Console.ReadKey();
    return 0;
}
```


© 2025 光子（深圳）精密 版权所有